$$S_n = 1 + t + t^2 + t^3 + \cdots t^{n-1} = \frac{1-t^n}{1-t}$$

$$S_\infty = 1 + t + t^2 + t^3 + \cdots \infty = \frac{1}{1-t}$$

$(t<1)$

$\frac{d}{dt}$ on both sides

$$0 + 1 + 2t + 3t^2 + 4t^3 + 5t^4 + \cdots \infty = \frac{d}{dt}\left(\frac{1}{(1-t)}\right)$$

$$= \frac{-1}{(1-t)^2} \times (-1)$$

$$= \frac{1}{(1-t)^2}$$

$$S = 1 + 2t + 3t^2 + 4t^3 + \cdots \infty \qquad (t<1)$$

$$tS = \quad t + 2t^2 + 3t^3 + 4t^4 + \cdots \infty$$

$$S - tS = 1 + t + t^2 + t^3 + \cdots \infty$$

$$S(1-t) = \frac{1}{(1-t)}$$

$$\Rightarrow \boxed{S = \frac{1}{(1-t)^2}}$$

$$\frac{d}{dx}\left(\frac{1}{x}\right) = \frac{d}{dx}x^{-1} = -1x^{-2} = \frac{-1}{x^2}$$

Amortized running time for insertLast

c_i - Cost of inserting the ith item

$$c_i = 1 + \begin{cases} i-1 & \text{if } i-1 = 2^k \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Amortized RT} = \frac{\sum_{i=1}^{n} c_i}{n} = \frac{\sum_{i=1}^{n} 1 + \sum_{j=1}^{\lfloor \log_2(n-1) \rfloor} 2^j}{n}$$

$$= \frac{n + 2\times\left(2^{\log_2(n-1)} - 1\right)}{n}$$

$$a^{\log_a n} = n$$

$$= \frac{n + 2(n-1-1)}{n} = \frac{n+2n-4}{n}$$

$$= \Theta(1)$$

n to $\log_2 n$

is 2^n to n

$16 = 2^4$ $\log_2 16 = \log_2 2^4 = 4\log_2 2 = 4$

$\div 2$ $\times 2$

$8 = 2^3$

$\div 2$ $\times 2$

$4 = 2^2$

$\div 2$ $\times 2$

$2 = 2^1$

$\div 2$ $\times 2$

$1 = 2^0$

$c_1 + c_2 + \dots + c_7$

$\downarrow$

$1 + 16$

$+0$ $+0$

$c_1 + c_2 + \dots + c_{16} + c_{17} + c_{18} + c_{19}$

powers of 2 $= \lfloor \log_2(19-1) \rfloor$

$2 + 4 + 8 + 16$

$2^1 + 2^2 + 2^3 + 2^4$

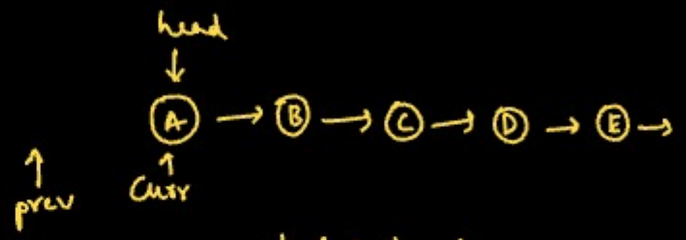
$$2^1 + 2^2 + 2^3 + \dots + 2^k = \frac{a(1-r^n)}{(1-r)} = \frac{2(1-2^k)}{(1-2)} = 2(2^k - 1)$$

$r = 2$

$a = 2$

$n = k$

- **Banker's method:**
  - Every element we insert gets 3 coins which could be used for doing work
    - 1 coin – work for raw insertion (at index i)
  - Keep 1 coin with the element & Give 2nd coin to the element at $i - cap/2$



start of iteration:
- we have curr & prev

Base case (each element has atleast one coin after its insertion)

a^0	b^0	c^0	d^0

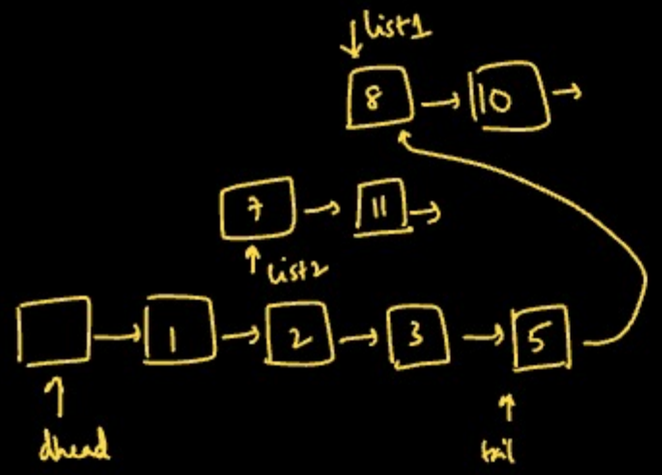
↓ resize (for copy of each element use its coin)

0	1	2	3	4	5	6	7
a	b	c	d				

cap = 8

↓ insert(e), e – 3 coins
- 1 coin for insertion of e
- 1 coin kept by e itself
- 1 coin to $i - \frac{cap}{2} = 4 - \frac{8}{2} = 0 \Rightarrow a$

a^0	b	c	d	e^0			



["cat", "dog" , "bat"]

- RT for checking if the list contains a string O(n)

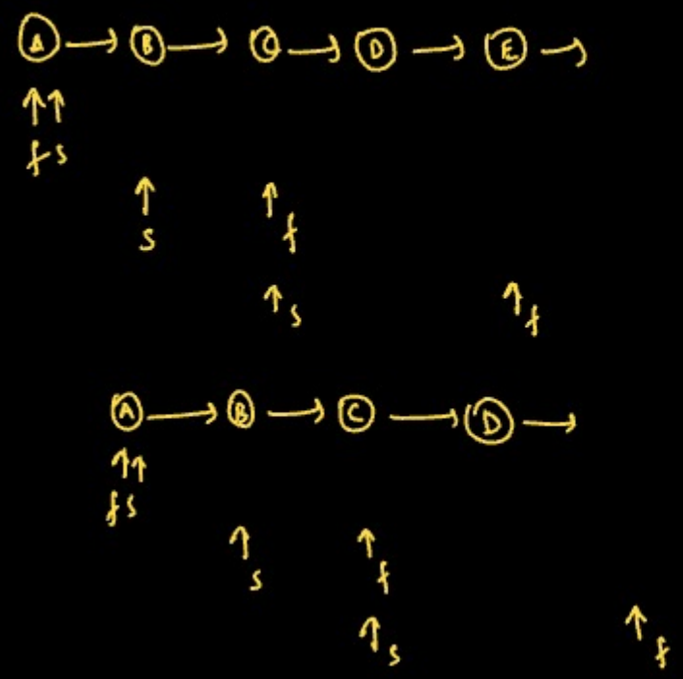
Set / HashSet / unordered_set → checking if the string exists in the set → O(1) RT on average.
(python) (Java) (C++)

keys

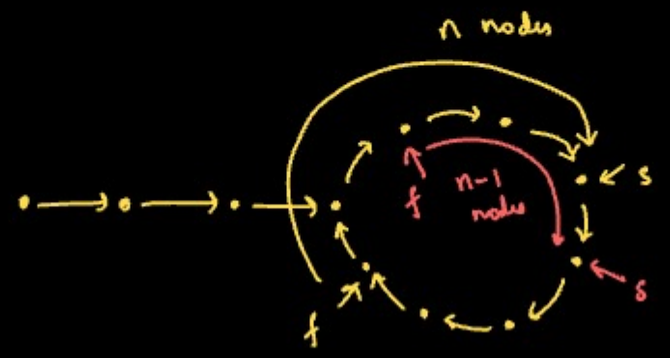
HashTables is the datastructure that enables average constant time lookup

Dictionary / HashMap / unordered_map → constant time for the following ops
(Python) (Java) (C++)

1. adding a new (k,v) pair
2. checking if 'k' already exists
3. updating value corresponding to a key.
4. retrieving value corresponding to a key




```
fast != null  && fast.next != null
while( fast.next != null && fast != null) {
    slow = slow.next ;
    fast = fast.next.next ;
}
```

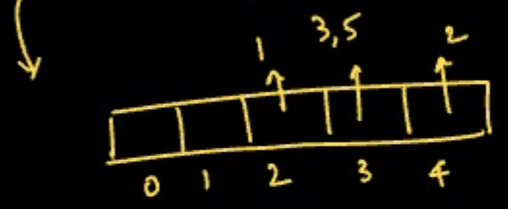


Observations:

1. Once a pointer (f/s) enters the cycle, it never exits.
2. Since it is a finite list, both pointers would enter the cycle.
3. The dist from f to s keeps reducing by 1 in every iteration. ∴ They have to meet.

[1, 2, 3, 3, 2, 2, 1, 3, 2, 5, 5, 5]

{1:2, 2:4, 3:3, 5:3}



1 → 2 → 3 → 4 → 5 →

1 → 2 → 3 → 4 → 5 →

5 → 4 → 3 →

1 → 2 → 3 → 4 → 5 →

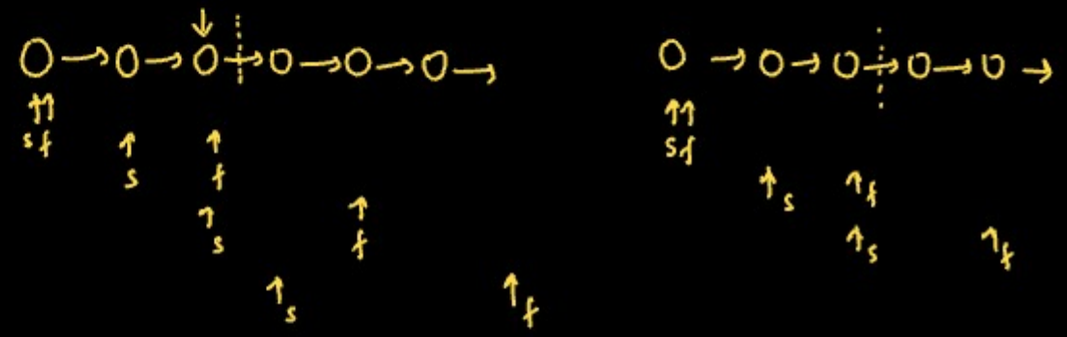
5 → 4 →

1 → 5 → 2 → 4 → 3 →

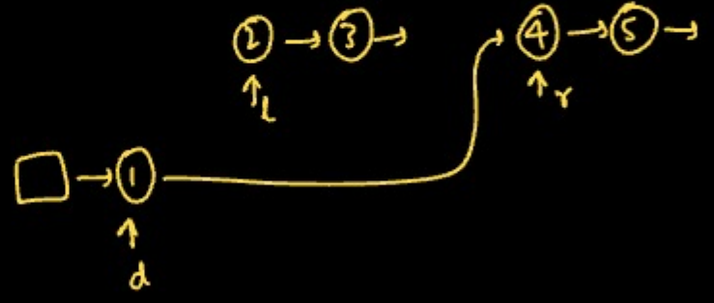
1 5 2 4 3

split:

- split list into 2 halves & return head of the right list.
- If odd #elements, left should have one extra
- We can assume atleast 3 nodes



while (fast.next != null && fast.next.next != null)



$O(n)$ - upper bound asymptotically

- constants & lower order terms don't matter

$f(n) = O(g(n))$ iff $\exists\, c, n_0$ s.t. $f(n) \le c \cdot g(n)$ $\forall n \ge n_0$

$3n^2+4n+6$ $\Leftrightarrow$ definition

$c \cdot g(n) \ge f(n) \quad \forall n \ge n_0$

- $f(n) = 3n^2+4n+6 \qquad f(n) = O(n^2)$

$3n^2 \le 3n^2 \quad \forall n \ge 1$
$4n \le 4n^2 \quad \forall n \ge 1$
$6 \le 6n^2 \quad \forall n \ge 1$

$3n^2+4n+6 \le 3n^2+4n^2+6n^2 = 13n^2$

$3n^2+4n+6 \le \underbrace{13}_{c} \cdot n^2 \quad \forall n \ge \underbrace{1}_{n_0} \Rightarrow \boxed{f(n) = O(n^2)}$

- $3n^2+4n+6 = O(n^3)$ ✓
  $= O(n^3 \log n)$ ✓
  $= O(n^2 \log n)$ ✓
  $= O(n \log n)$ ✗

$\log_6 n = \dfrac{\log_{10} n}{\log_{10} 6}$

$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec \dots \prec n^{1000} \prec 1.1^n$

$1.1^n = O(n^{1000})$

- $\Omega(n)$ - lower bound
  - $f(n) = \Omega(g(n))$ iff $\exists\, c, n_0$ s.t. $f(n) \ge c \cdot g(n)$ $\forall n \ge n_0$

$3n^2+4n+6$: $\Omega(n^2)$ ✓
$\Omega(n^2 \log n)$ ✗
$\Omega(n)$ ✓
$\Omega(n \log n)$ ✓
$\Omega(n^3)$ ✗

$3n^2 \ge n^2 \quad \forall n \ge 1$
$3n^2+4n+6 \ge n^2 \quad \forall n \ge 1$
$\exists\, c = 1, n_0 = 1$
$\Rightarrow f(n) = \Omega(n^2)$

Orders of growth
1
n
$\log n$
n^2
n^3
$n \log n$
$\vdots$

- Θ :

$f(n) = \Theta(g(n))$ iff $\exists\, c_1, c_2, n_0$ $c_1 g(n) \le f(n) \le c_2 g(n)$ $\forall n \ge n_0$

$f(n) = \Theta(g(n))$ iff $[f(n) = O(g(n)) \;\&\; f(n) = \Omega(g(n))]$

	proc 1	proc 2 (doubly powerful)
$O(n)$	n_0	$2n_0$
$O(n^2)$	n_0	$\sqrt{2}\, n_0$
$O(n^3)$	n_0	$\sqrt[3]{2}\, n_0$
$O(2^n)$	n_0	n_0+1

$2^{n_0+1} = 2 \cdot 2^{n_0}$

- Code examples – running time
- Master Theorem & recurrences.

Eg:

```
for(i=0; i<n; i++)
    sum = sum + i;
```

$\Theta(n)$ as each loop body is constant time

```
for(i=1; i≤n; i=2i)
    // Constant op
```

This loop runs $\sim \lg n$ times

$\Rightarrow$ RT of loop $= \Theta(\log n)$

```
for(i=n; i>0; i=i/3){
    // Θ(n) operation
}
```

This loop runs $\sim \log_3 n$ times

$\Rightarrow RT = \Theta(n \log n)$

```
for(i=0; i<n; i++){
    for(j=0; j<n; j++){
        // Constant work
    }
}
```

$\Theta(n)$ op

$RT = \Theta(n^2)$

```
for(i=0; i<n; i++){
    for(j=i+1; j<n; j++){
        // constant op - k
    }
}
```

	inner loop work
$i=0$	$k(n-1)$
$i=1$	$k(n-2)$
$\vdots$	$\vdots$
$i=n-2$	$k(1)$
$i=n-1$	0

Total work $= k(1+2+3+\cdots+(n-1))$

$= \frac{k(n(n-1))}{2} = \frac{k}{2}n^2 - \frac{k}{2}n = \Theta(n^2)$

$S_n = \frac{n(n+1)}{2}$

base 2

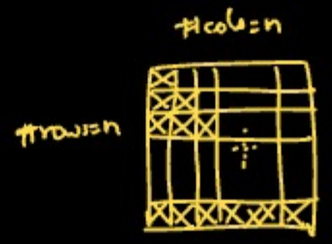
$n \xrightarrow{\div 2} n/2 \xrightarrow{\div 2} n/4 \xrightarrow{\div 2} \cdots \xrightarrow{\div 2} 2 \xrightarrow{\div 2} 1$

$16 = 2^4 \xrightarrow{\div 2} 8 = 2^3 \xrightarrow{\div 2} 4 = 2^2 \xrightarrow{\div 2} 2 = 2^1 \xrightarrow{\div 2} 1 = 2^0$

$\times 2$

$\boxed{\log_a a^b = b}$

$\log_2 16 = \log_2 2^4 = 4$



Total #squares $= n^2$

$(n) + 2(\underbrace{1+2+\cdots+n-1}) = n^2$

$S = 1 + 2 + 3 + \cdots + (n-1) + n$

$\boxed{S \le n^2}$

$\frac{1}{2}$ of the terms $\ge \frac{n}{2}$ $\Rightarrow S \ge \frac{n}{2}\left(\frac{n}{2}\right) \Rightarrow \boxed{S \ge \frac{1}{4}n^2}$

terms $\ge n/2$

Master theorem

```
mergesort(array): if(array.len = 0 or 1) return array;
    left : array[:n/2]
    right = array[n/2:]
    { sorted_left: mergesort(left)
    { sorted_right: mergesort(right)

    sorted = merge(sorted_left, sorted_right)   // Θ(n) time
    return sorted                                  n - array's length
```

$T(n)$:= time taken for mergesort on 'n' sized array

$T(1) = 1$

$$T(n) = \underbrace{T(n/2)}_{\text{work done for merge sort on left half}} + \underbrace{T(n/2)}_{\text{work done for merge sort on right half}} + kn$$

kn → work done outside the recursive calls (merge, checking base case, splitting array ...)

$$\boxed{T(n) = 2T(n/2) + kn}$$

→ Recurrence relation for merge sort

Master Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + O(n^d)$$

a = # recursive calls

b = factor of subproblem size compared to the problem size

n^d - work done outside recursion

3 Cases :

Case 1: $a = b^d \Rightarrow RT = O(n^d \log n)$

Case 2: $a < b^d \Rightarrow RT = O(n^d)$

Case 3: $a > b^d \Rightarrow RT = O(n^{\log_b a})$

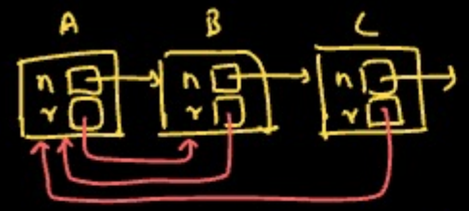
- Mergesort:

  $a = 2$, $b = 2$, $d = 1$ $\Rightarrow$ Case 1 $\Rightarrow RT = O(n^d \log n)$
  $= O(n \log n)$

- Binary search

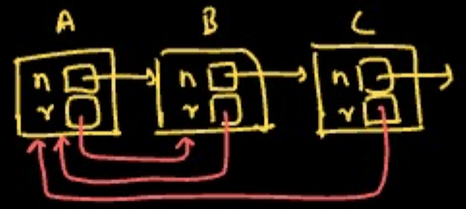
  Recurrence relation: $T(n) = 1 \cdot T(n/2) + O(1)$

  $a = 1$, $b = 2$, $d = 0$ $\Rightarrow$ Case 1 $\Rightarrow RT = O(n^d \log n)$
  $= O(\log n)$

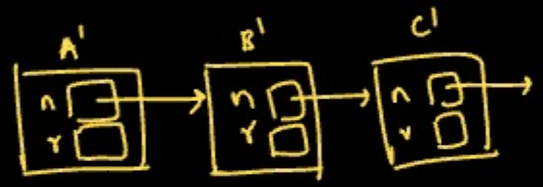


Hash map

Key	Value
A	A'
B	B'
C	C'

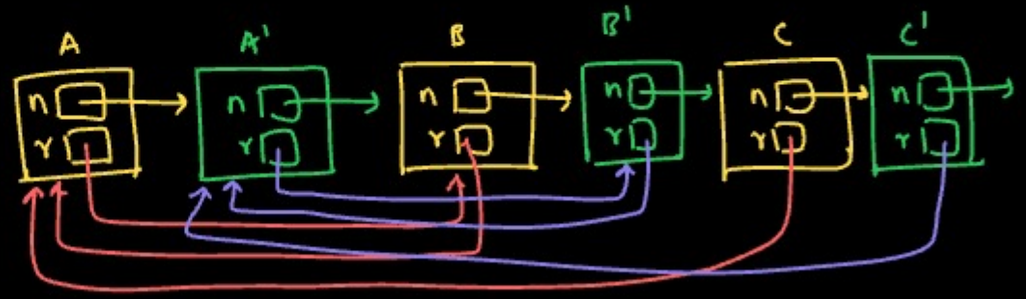


Copy



for each curr in original list

map[curr].random = map[curr.random]



Merge k sorted lists:

- Approach 1: Merge each list one by one into an accumulator list

k - # lists, n - # nodes per list.

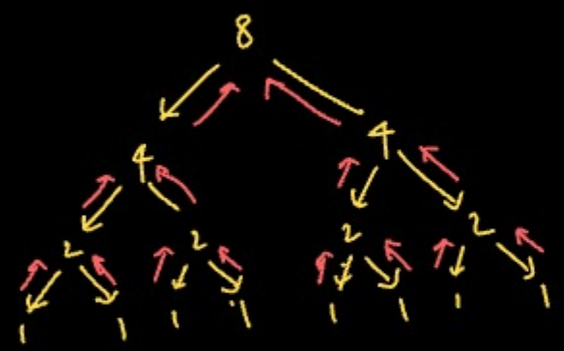
	# steps
first merge	$c(2n)$
second merge	$c(3n)$
⋮	
last merge	$c(kn)$

$$RT = cn(2+3+\cdots+k)$$
$$= cn\left(\frac{k(k+1)}{2} - 1\right)$$
$$= \Theta(nk^2)$$

merge(list 1, list 2)
↓ m nodes ↓ n nodes

$\boxed{RT = \Theta(m+n)}$

- $m = \Theta(n) \Rightarrow RT = \Theta(m)$ or $\Theta(n)$
- $m >> n$
  (Eg: $n = \log m$) $\Rightarrow RT = \Theta(m)$



$\boxed{k = 2^p} \Rightarrow p = \log_2 k$

	# merge operations	work done per merge	Total work done
first pass	$\frac{k}{2} = 2^{p-1}$	$2n$	$2^{p-1} \times 2n = 2^p n$
second pass	$\frac{k}{4} = 2^{p-2}$	$4n$	$2^{p-2} \times 2^2 n = 2^p n$
third pass	$\frac{k}{8} = 2^{p-3}$	$8n$	$2^{p-3} \times 2^3 n = 2^p n$
⋮	⋮	⋮	
last pass	$1 = 2^0$	$nk = 2^p n$	$2^p n$

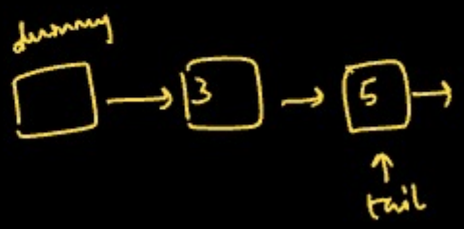
$\Rightarrow$ Total work = # passes $\times$ (work per pass) $= (\log_2 k)\, 2^p n = nk \log k = \Theta(nk \log k)$

- Approach 4: Create mega list of all lists : nk steps (connect tail of each list to head of next)

  - Sort the mega list : $\underbrace{\Theta(nk \log(nk))}_{\text{Total RT}}$

Priority Queue interface

Worst case

	removeMin()	getMin()	addNew()
Sorted array	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
Unsorted array	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Sorted linked list	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
Heaps	$\boxed{\Theta(\log n)}$	$\Theta(1)$	$\boxed{\Theta(\log n)}$

Min Heap (k elements)